TCP Vs QUIC Under Wireless-like Random Packet Loss

EE6750 - Transport Over Wireless Networks

3/6/2026

- Team Members & Roles:
  1. Raghuvamsi Lekkala: Implementation, experiments, data collection
  2. Krishna Sri Mannam: Experiment design, analysis, report writing
- Tools We Are Using:
  i. WSL2 + tc netem: Simulating random wireless packet loss on Windows (craigloewen-msft, 2024).
  ii. aioquic: Running QUIC protocol (Python library) (Lainé, 2026).
  iii. Python sockets/iperf3: Running TCP for comparison (GUEANT, 2013)
  iv. Wireshark: Capturing and inspecting packets (Wireshark Foundation, n.d.)
  v. Python (matplotlib): Plotting graphs and analyzing results (Python, 2025)

Problem Statement & Research Question

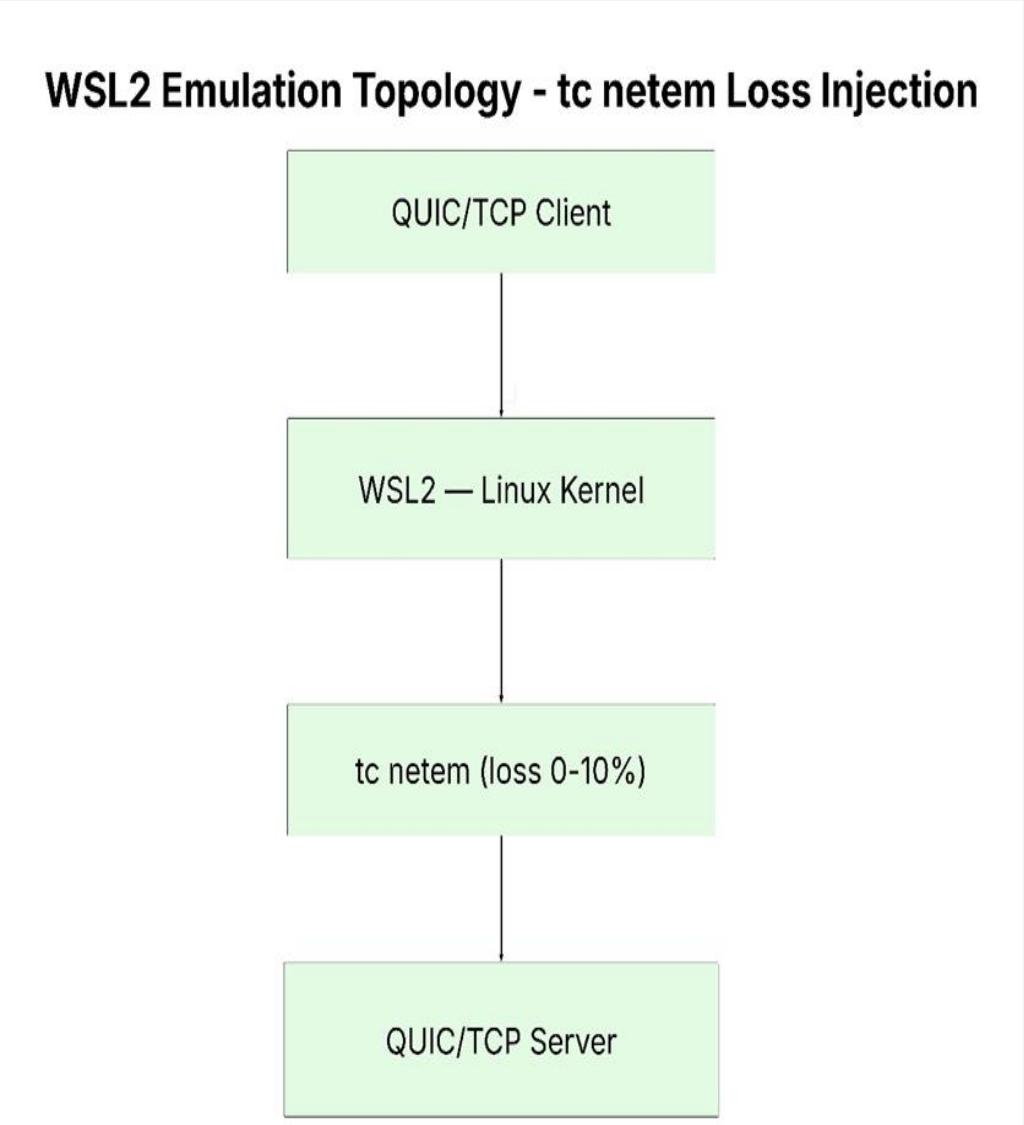
- Problem:
  - ✓ Wireless networks drop packets due to interference, not congestion.
  - ✓ TCP misinterprets packet loss as congestion, slowing down unnecessarily (Arghavani, 2026).
  - ✓ QUIC was designed to handle loss more intelligently (Ghedini, 2018).
- Research Question:
  - ✓ How does QUIC compare to TCP in throughput and flow completion time when transferring a 10MB file under random wireless-like packet loss rates of 0%, 1%, 3%, 5%, and 10%?

Why It Matters

- Real-World Impact:
  - ✓ Most internet traffic runs over WiFi or mobile networks, yet TCP was designed for wired networks in the 1970s.
  - ✓ Users experience slow downloads and buffering on uncongested WiFi.
- Who is Already Using QUIC?
  - ✓ Google, YouTube, HTTP/3: Faster page loads, reduced buffering, better mobile performance.
  - ✓ Meta/Facebook: Improved mobile network performance.
- Why This Research Matters:
  1. We need data: Exact figures on TCP degradation vs QUIC under real loss conditions.
  2. QUIC is not always available: Some firewalls block UDP, forcing TCP use.
  3. Network engineers need this: To make informed protocol decisions and justify infrastructure upgrades.

Wireless Mechanisms

- What Actually Happens in a Wireless Network?
  - ✓ Physical Layer: Loss due to radio interference and noise.
  - ✓ Link Layer: WiFi MAC retries failed frames (4-7 attempts), then drops the frame.
  - ✓ Transport Layer: Both TCP and QUIC see packet loss, but react differently.
- Simulating with tc netem:
  - ✓ tc qdisc add dev eth0 root netem loss 5% - Simulates random packet loss.



Layered Reasoning Map

- What Happens at Each Layer:
  - ✓ Physical Layer: Random bit errors due to interference.
  - ✓ Link Layer: WiFi retries failed frames.
  - ✓ Network Layer: Both TCP and QUIC detect lost packets but react differently.
  - ✓ Transport Layer:
    - TCP: Assumes loss is due to congestion, halves CWND (Congestion Window).
    - QUIC: Selective retransmit, minimal CWND reduction.
  - ✓ Application Layer:
    - TCP: Slower file transfer.
    - QUIC: Faster, smoother file transfer.
- Key Insight:
  - ✓ TCP misinterprets packet loss as congestion, slowing down unnecessarily.
  - ✓ QUIC handles loss more efficiently, maintaining better throughput.

Hypotheses

H1 - Throughput:

- Hypothesis: QUIC will maintain higher throughput than TCP as packet loss increases.
- Reasoning: TCP's congestion window halves with each loss, while QUIC maintains a more consistent sending rate.

H2 - Flow Completion Time (FCT):

- Hypothesis: QUIC will complete the file transfer faster and more consistently than TCP.
- Reasoning: QUIC recovers faster, while TCP's repeated slowdowns result in longer transfers.

H3 - Retransmissions:

- Hypothesis: TCP's retransmission count will increase faster than QUIC's.
- Reasoning: TCP retransmits more data unnecessarily, while QUIC only retransmits lost packets.

Experimental Design

- Method: Emulation
  - ✓ Emulation allows for real protocol behavior in a controlled, repeatable environment.
  - ✓ tc netem is used for packet loss simulation on Windows using WSL2 (Kerrisk, n.d.).
- Independent Variable:
  - ✓ Loss Rate: 0%, 1%, 3%, 5%, 10%.
- Metrics Measured:
  - ✓ Goodput: Mbps
  - ✓ Flow Completion Time (FCT): ms
  - ✓ Retransmissions: Count
  - ✓ Latency CDF: ms
- Baseline vs Comparison:
  - ✓ Baseline: TCP Cubic
  - ✓ Comparison: QUIC (aioquic)
- Core Scenarios:
  - ✓ Scenario 1: No loss (0%)
  - ✓ Scenario 2: Medium loss (3%)
  - ✓ Scenario 3: High loss (10%)

Implementation Progress

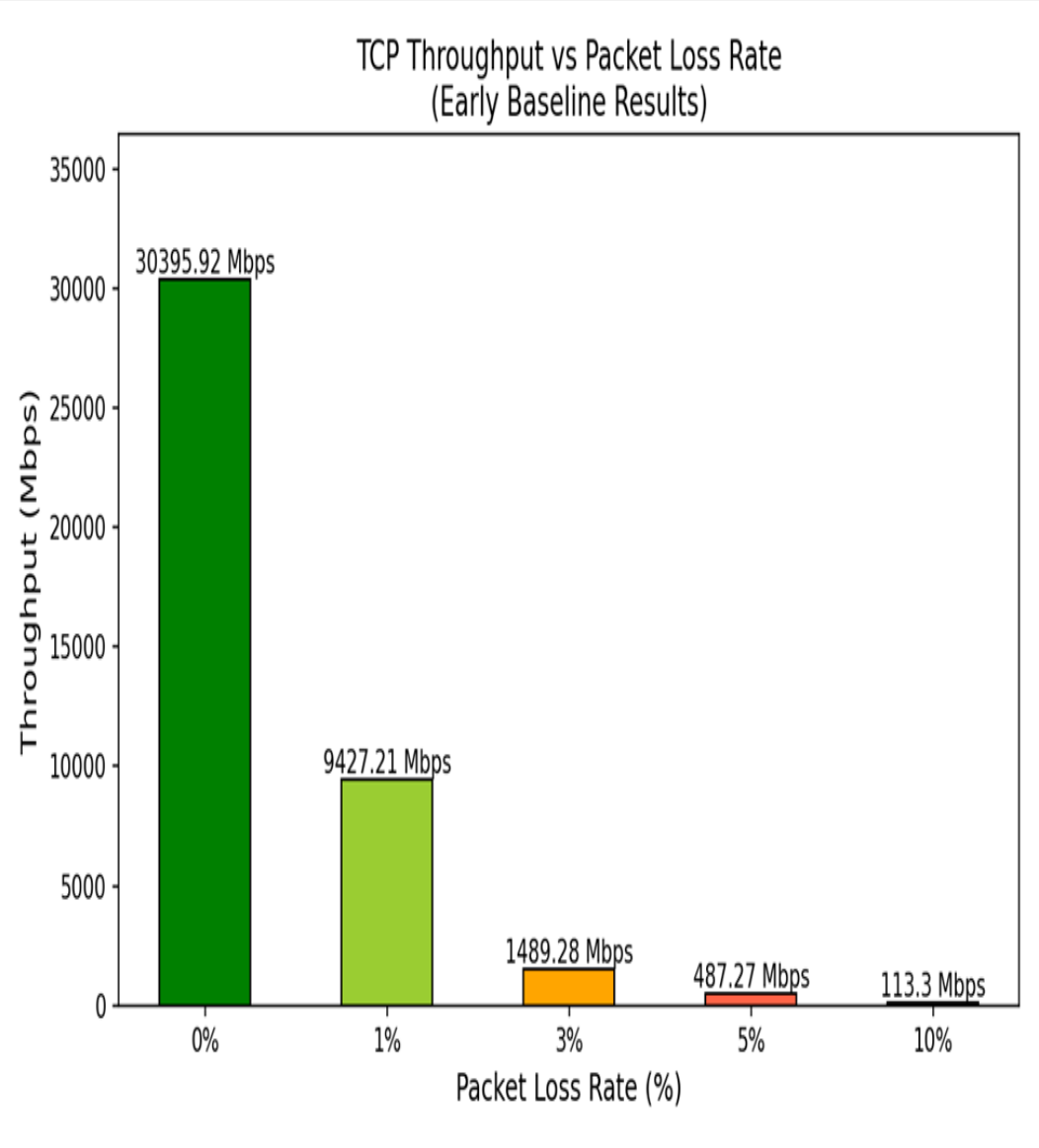
- What We Have Done So Far:
  - ✓ WSL2 Environment Setup: Completed and tested with necessary tools installed (aioquic, iperf3, tc netem, python).
  - ✓ Loss Injection Ready: Tested packet loss injection with tc netem.
  - ✓ GitHub Repository: Set up with all scripts and experiment details documented.
- Next Steps:
  - ✓ Week 1: Finalize aioquic server and client, collect first QUIC results.
  - ✓ Week 2: Full data collection (100 runs).
  - ✓ Week 3: Analysis, CDF plots, and comparison.
  - ✓ Week 4: Final report and presentation.

```
4.whl (219 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 219.6/219.6 kB 1.3 MB/s eta 0:00
:00
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.6/44.6 kB 836.8 kB/s eta 0:00
:00
Downloading pyasn1-0.6.2-py3-none-any.whl (83 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 83.4/83.4 kB 897.5 kB/s eta 0:00
:00
Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 181.3/181.3 kB 1.3 MB/s eta 0:00
:00
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48.2/48.2 kB 757.2 kB/s eta 0:00
:00
Installing collected packages: typing-extensions, pylsqpack, pycparser, pyas
n1, certifi, attrs, pyasn1-modules, cffi, cryptography, service-identity, py
openssl, aioquic
Successfully installed aioquic-1.3.0 attrs-25.4.0 certifi-2026.2.25 cffi-2.0
.0 cryptography-46.0.5 pyasn1-0.6.2 pyasn1-modules-0.4.2 pycparser-3.0 pylsq
pack-0.3.23 pyopenssl-25.3.0 service-identity-24.2.0 typing-extensions-4.15.
0
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ python3 -c "import aioquic;
 print('aioquic working!')"
-bash: !': event not found
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ sudo tc qdisc add dev lo ro
ot netem loss 0%
iperf3 -c 127.0.0.1 -t 10 -J > result_0.json
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ sudo tc qdisc change dev lo
 root netem loss 1%
iperf3 -c 127.0.0.1 -t 10 -J > result_1.json
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ sudo tc qdisc change dev lo
 root netem loss 3%
iperf3 -c 127.0.0.1 -t 10 -J > result_3.json
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ sudo tc qdisc change dev lo
 root netem loss 5%
iperf3 -c 127.0.0.1 -t 10 -J > result_5.json
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ sudo tc qdisc change dev lo
 root netem loss 10%
iperf3 -c 127.0.0.1 -t 10 -J > result_10.json
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$
```

```
adminlinux@DESKTOP-TTFHH9K:~$ cd tcp-vs-quic
adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ source venv/bin/activate
(venv) adminlinux@DESKTOP-TTFHH9K:~/tcp-vs-quic$ iperf3 -s
-----------------------------------------------------------
Server listening on 5201 (test #1)
-----------------------------------------------------------
Accepted connection from 127.0.0.1, port 52128
[  5] local 127.0.0.1 port 5201 connected to 127.0.0.1 port 52142
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-1.00   sec  3.45 GBytes  29.6 Gbits/sec
[  5]   1.00-2.00   sec  3.74 GBytes  32.1 Gbits/sec
[  5]   2.00-3.00   sec  3.70 GBytes  31.8 Gbits/sec
[  5]   3.00-4.00   sec  3.56 GBytes  30.6 Gbits/sec
[  5]   4.00-5.00   sec  3.47 GBytes  29.8 Gbits/sec
[  5]   5.00-6.00   sec  3.06 GBytes  26.3 Gbits/sec
[  5]   6.00-7.00   sec  3.57 GBytes  30.6 Gbits/sec
[  5]   7.00-8.00   sec  3.77 GBytes  32.4 Gbits/sec
[  5]   8.00-9.00   sec  3.56 GBytes  30.6 Gbits/sec
[  5]   9.00-10.00  sec  3.54 GBytes  30.4 Gbits/sec
[  5]  10.00-10.01  sec  1.62 MBytes  1.64 Gbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-10.01  sec  35.4 GBytes  30.4 Gbits/sec                  receiv
er
-----------------------------------------------------------
Server listening on 5201 (test #2)
-----------------------------------------------------------
Accepted connection from 127.0.0.1, port 49868
[  5] local 127.0.0.1 port 5201 connected to 127.0.0.1 port 49882
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-1.00   sec   684 MBytes  5.73 Gbits/sec
[  5]   1.00-2.00   sec  1.45 GBytes  12.5 Gbits/sec
[  5]   2.00-3.00   sec  1.53 GBytes  13.2 Gbits/sec
[  5]   3.00-4.00   sec  1.17 GBytes  10.1 Gbits/sec
[  5]   4.00-5.00   sec   871 MBytes  7.30 Gbits/sec
[  5]   5.00-6.00   sec   779 MBytes  6.54 Gbits/sec
[  5]   6.00-7.00   sec   816 MBytes  6.84 Gbits/sec
[  5]   7.00-8.00   sec  1.17 GBytes  10.1 Gbits/sec
[  5]   8.00-9.00   sec  1.32 GBytes  11.3 Gbits/sec
[  5]   9.00-10.00  sec  1.25 GBytes  10.7 Gbits/sec
[  5]  10.00-10.00  sec   896 KBytes  14.0 Gbits/sec
```

Early Results

- TCP Baseline Results (with 5 loss rates):
  - ✓ 0%: 30,395 Mbps (baseline)
  - ✓ 1%: 9,427 Mbps (↓ 69%)
  - ✓ 3%: 1,489 Mbps (↓ 95%)
  - ✓ 5%: 487 Mbps (↓ 98%)
  - ✓ 10%: 113 Mbps (↓ 99.6%)
- Key Observations:
  1. Exponential degradation: Even 1% packet loss causes a 70% drop in throughput.
  2. TCP's vulnerability: Performance collapses as packet loss increases, especially at 10%.
  3. Supports our hypothesis: TCP is highly sensitive to random packet loss.
- Next Step:
  - ✓ To run QUIC side-by-side and compare performance.



Rubric Alignment And Schedule

- Rubric Alignment:
  - ✓ Problem & Motivation: Clearly defined with real-world impact.
  - ✓ Course Concepts: Full 5-layer reasoning from Physical to Application layer.
  - ✓ Experimental Design: 5 loss rates, 10 trials each, 100 runs, CDFs and confidence intervals planned.
  - ✓ Implementation Status: WSL2 environment working, baseline experiments completed.
  - ✓ Results & Analysis: Early data collected and visualized.
  - ✓ Reproducibility: GitHub repo live, all scripts and commands documented.
- Updated Schedule:
  - ✓ Week 1: Finalize QUIC setup, verify results.
  - ✓ Week 2: Full data collection (100 runs).
  - ✓ Week 3: Analysis, graphs, CDFs.
  - ✓ Week 4: Report, presentation, final submission.

References

Arghavani, M. (2026). *Optimising TCP slow-start: in-depth analysis and enhanced designs*. Otago.ac.nz. https://ourarchive.otago.ac.nz/esploro/outputs/doctoral/Optimising-TCP-slow-start-in-depth-analysis-and/9926545580001891?institution=64OTAGO_INST

craigloewen-msft. (2024). *Install WSL*. Learn.microsoft.com. https://learn.microsoft.com/en-us/windows/wsl/install

Ghedini, A. (2018). *The Road to QUIC*. The Cloudflare Blog. https://blog.cloudflare.com/the-road-to-quic/

GUEANT, V. (2013). *iPerf - The TCP, UDP and SCTP network bandwidth measurement tool*. Iperf.fr. https://iperf.fr/

Kerrisk, M. (n.d.). *tc-netem(8) - Linux manual page*. Man7.org. https://man7.org/linux/man-pages/man8/tc-netem.8.html

Lainé, J. (2026). *aioquic — aioquic documentation*. Readthedocs.io. https://aioquic.readthedocs.io/en/latest/

Python. (2025). *Python*. Python.org; Python.org. https://www.python.org/

Wireshark Foundtion. (n.d.). *Wireshark*. Wireshark.org. https://www.wireshark.org/